KDT AI 캠퍼스 8차시 마스터 커리큘럼 & 실습 교안

주제: [Python] OOP(클래스) 및 예외처리 실습, [Codex] 기반 코드 개선 및 리팩토링 전후 비교 분석 | 검증 기준: Python 3.10+ 무결성(Zero-Error) 검증 완료

전체 8차시 커리큘럼 로드맵: 기초 설계부터 AI 리팩토링까지 단계별 마스터

본 교안은 KDT AI 캠퍼스 수강생이 객체지향 프로그래밍(OOP)과 예외처리의 핵심 원리를 직관적인 비유로 체득하고, 실무에서 Codex(AI 코드 어시스턴트)를 활용해 레거시 스파게티 코드를 견고한 프로덕션급 파이프라인으로 리팩토링할 수 있도록 설계된 8차시(총 8시간) 통합 과정입니다. 수록된 모든 실습 코드는 실제 파이썬 런타임에서 문법 및 실행 오류 제로(0건)로 검증되었습니다.

차시	모듈 구분	핵심 학습 목표 (선언적 성취 기준)	핵심 문법 키워드	실습 산출물 예제
1차시	OOP 입문과 탄생	클래스와 인스턴스로 데이터와 행동을 하나의 설계도로 통합한다	`class`, `__init__`, `self`	KDT 수강생 성장기 & 스마트 바리스타 로봇
2차시	캡슐화와 보안	캡슐화와 프로퍼티로 객체 내부 데이터의 무결성을 보호한다	`__private`, `@property`, `__str__`	디지털 금고 & AI 하이퍼파라미터 검증기
3차시	상속과 다형성	상속과 다형성으로 중복을 제거하고 추론 인터페이스를 표준화한다	`super()`, 오버라이딩, 다형성	멀티모달 AI 모델(Vision/LLM) 추론 파이프라인
4차시	모던 파이썬 OOP	추상 클래스와 데이터클래스로 엔터프라이즈 설계 규격을 강제한다	`ABC`, `@abstractmethod`, `@dataclass`	다중 결제 게이트웨이 및 자동 정산 영수증
5차시	예외처리 4단계	4단계 안전망으로 비정상 입력에도 멈추지 않는 로거를 구현한다	`try`, `except`, `else`, `finally`	오류 라인을 격리하는 무중단 AI 학습 로그 파서
6차시	커스텀 예외 설계	도메인 맞춤 예외 클래스와 `raise`로 비즈니스 규칙을 수호한다	`Exception` 상속, `raise`	Codex API 토큰 쿼터 및 프롬프트 보안 제어기
7차시	Codex 리팩토링 1	Codex 프롬프트로 절차지향 조건문을 클린 OOP 아키텍처로 전환한다	매직 넘버 제거, 불변 객체, 타입 힌트	온라인 서점 할인 계산기 Before & After
8차시	Codex 리팩토링 2	취약한 AI 전처리 함수를 방어적 OOP 파이프라인으로 리팩토링한다	예외 체이닝(`from exc`), 리포트 객체	AI 점수 정규화 파이프라인 Before & After 캡스톤

1차시: 클래스(Class)와 객체(Object)의 탄생 — 붕어빵 틀을 넘어 '스마트 캐릭터 공장'으로!

1. 재미있는 문법 스토리텔링: 왜 딕셔너리와 함수만으로는 부족할까?

요리사가 주방에서 재료 봉투(딕셔너리)와 레시피 쪽지(함수)를 따로 던져두면, 주문이 100개로 늘어났을 때 어느 손님의 재료인지 뒤섞여 대참사가 일어납니다. **객체지향 프로그래밍(OOP)**은 **데이터(상태, Attribute)**와 **기능(행동, Method)**을 하나의 '스마트 로봇' 안에 캡슐처럼 합쳐 놓는 기술입니다.

- **`class` (마스터 설계도)**: 로봇의 속성과 버튼을 정의한 청사진입니다. 설계도 자체는 일하지 않으며, 설계도로부터 실체(인스턴스)를 찍어내야 움직입니다.
- **`__init__` (출생 신고서 마법 메서드)**: 로봇이 공장에서 갓 조립되어 나올 때 **단 한 번 자동으로 호출**되며, 이름·초기 체력·트랙 같은 초기 속성을 세팅합니다.
- **`self` (나 자신을 가리키는 명찰)**: 똑같은 설계도로 만든 100명의 수강생 객체 중 "지금 커피를 마시고 레벨업하는 바로 나 자신"의 메모리 주소를 가리키는 필수 첫 번째 매개변수입니다.

2. 실습 예제 1-1 (기초): KDT AI 캠퍼스 수강생 성장 시뮬레이터

```
class KDTStudent:
    def __init__(self, name: str, track: str):
        self.name = name
        self.track = track
        self.level = 1
        self.energy = 100
        self.completed_missions = []

    def study_coding(self, topic: str, cost_energy: int = 25) -> str:
        if self.energy < cost_energy:
            return f"[휴식 필요] {self.name}님의 에너지가 부족합니다! (현재: {self.energy})"
        self.energy -= cost_energy
        self.level += 1
        self.completed_missions.append(topic)
        return f"[레벨업!] {self.name}님이 '{topic}' 실습을 완료했습니다! (Lv.{self.level}, 남은 에너지: {self.energy})"

    def drink_coffee(self, boost: int = 40) -> str:
        self.energy = min(100, self.energy + boost)
        return f"[충전 완료] 아이스 아메리카노 수혈! 현재 에너지: {self.energy}"

student1 = KDTStudent("김루키", "AI 엔지니어링")
print(student1.study_coding("Python 클래스 기초"))
print(student1.study_coding("예외처리 마스터"))
print(student1.drink_coffee())
# 실행 결과:
# [레벨업!] 김루키님이 'Python 클래스 기초' 실습을 완료했습니다! (Lv.2, 남은 에너지: 75)
# [레벨업!] 김루키님이 '예외처리 마스터' 실습을 완료했습니다! (Lv.3, 남은 에너지: 50)
# [충전 완료] 아이스 아메리카노 수혈! 현재 에너지: 90
```

3. 실습 예제 1-2 (응용): 원두 재고를 스스로 관리하는 스마트 바리스타 로봇

```
class BaristaBot:
    def __init__(self, bot_name: str, bean_grams: int):
```

```
        self.bot_name = bot_name
        self.bean_grams = bean_grams
        self.total_cups = 0

    def brew_espresso(self, cups: int = 1) -> str:
        required_beans = cups * 18
        if self.bean_grams < required_beans:
            return f"[{self.bot_name}] 원두 부족! (필요: {required_beans}g, 현재:
{self.bean_grams}g)"
        self.bean_grams -= required_beans
        self.total_cups += cups
        return f"[{self.bot_name}] 에스프레소 {cups}잔 추출 완료! (남은 원두:
{self.bean_grams}g, 누적: {self.total_cups}잔)"

bot = BaristaBot("코덱스바리스타-1호", 50)
print(bot.brew_espresso(2))
print(bot.brew_espresso(2))
# 실행 결과:
# [코덱스바리스타-1호] 에스프레소 2잔 추출 완료! (남은 원두: 14g, 누적: 2잔)
# [코덱스바리스타-1호] 원두 부족! (필요: 36g, 현재: 14g)
```

2차시: 캡슐화(Encapsulation)와 프로퍼티(@property) — 내 지갑과 AI 가중치는 아무나 못 건드려!

1. 재미있는 문법 스토리텔링: 금고 잠금장치(__)와 스마트 문지기(@property)

만약 은행 앱에서 누군가 `wallet.balance = -999999`처럼 내 통장 잔고를 직접 수정할 수 있다면 어떨까요? 또는 딥러닝 모델의 학습률(Learning Rate)에 실수로 `-50.0`을 대입해 학습이 폭발한다면 큰일입니다. 그래서 중요한 내부 데이터는 **금고(Private 변수 `__변수명`)**에 숨기고, 검증 로직을 갖춘 **스마트 문지기(`@property`와 `@setter`)**를 통해서만 출입을 허락합니다.

- **`self.__balance` (맹글링 금고)**: 변수 앞에 언더바 두 개(__)를 붙이면 클래스 외부에서 직접 접근이 차단됩니다.
- **`@property` (안전한 조회 창구)**: 메서드이지만 외부에서는 일반 변수처럼(`wallet.balance`) 깔끔하게 읽을 수 있게 해줍니다.
- **`@balance.setter` (검문소 문지기)**: 값을 대입할 때(`wallet.balance = 값`) 자동으로 호출되어 음수나 비정상 범위를 걸러냅니다.

2. 실습 예제 2-1 (기초): 음수 조작을 원천 차단하는 해킹 방지 디지털 금고

```
class DigitalWallet:
    def __init__(self, owner: str, initial_balance: int = 0):
        self.owner = owner
        self.__balance = 0
        self.balance = initial_balance  # setter 검문소를 거쳐 안전하게 초기화

    @property
    def balance(self) -> int:
        return self.__balance
```

```
    @balance.setter
    def balance(self, amount: int) -> None:
        if amount < 0:
            print(f"[보안 경고] 잔액은 음수가 될 수 없습니다! (입력값: {amount}원 -> 거부됨)")
            return
        self.__balance = amount

    def deposit(self, amount: int) -> str:
        if amount <= 0:
            return "[오류] 입금액은 0원보다 커야 합니다."
        self.balance += amount
        return f"[{self.owner}] {amount:,}원 입금 완료 (현재 잔액: {self.balance:,}원)"

wallet = DigitalWallet("이코딩", 10000)
print(wallet.deposit(5000))
wallet.balance = -99999  # 외부에서 불법 음수 대입 시도 -> setter가 즉시 차단!
print(f"최종 안전 잔액: {wallet.balance:,}원")
# 실행 결과:
# [이코딩] 5,000원 입금 완료 (현재 잔액: 15,000원)
# [보안 경고] 잔액은 음수가 될 수 없습니다! (입력값: -99999원 -> 거부됨)
# 최종 안전 잔액: 15,000원
```

3. 실습 예제 2-2 (응용): AI 모델 하이퍼파라미터 컨트롤러와 __str__ 매직 메서드

```
class ModelConfig:
    def __init__(self, model_name: str, learning_rate: float, epochs: int):
        self.model_name = model_name
        self.__learning_rate = 0.001
        self.learning_rate = learning_rate
        self.epochs = epochs

    @property
    def learning_rate(self) -> float:
        return self.__learning_rate

    @learning_rate.setter
    def learning_rate(self, value: float) -> None:
        if not (0.0 < value <= 1.0):
            print(f"[검증 경고] 비정상 학습률({value}) 감지! 기존 안전값({self.__learning_rate})
을 유지합니다.")
            return
        self.__learning_rate = value

    def __str__(self) -> str:
        return f"ModelConfig(name='{self.model_name}', lr={self.learning_rate}, epochs=
{self.epochs})"

cfg = ModelConfig("ResNet-50", 0.01, 30)
print(cfg)
cfg.learning_rate = 5.0  # 허용 범위(0~1.0) 초과 대입 시도
print(cfg)
# 실행 결과:
# ModelConfig(name='ResNet-50', lr=0.01, epochs=30)
```

```
# [검증 경고] 비정상 학습률(5.0) 감지! 기존 안전값(0.01)을 유지합니다.
# ModelConfig(name='ResNet-50', lr=0.01, epochs=30)
```

3차시: 상속(Inheritance)과 다형성(Polymorphism) — 거인의 어깨 위에서 코드 재사용하기!

1. 재미있는 문법 스토리텔링: 부모 설계도 물려받기(super)와 만능 리모컨 버튼(다형성)

비전 AI 모델과 거대언어모델(LLM)을 만들 때마다 모델 이름, 버전 기록, 로그 출력 코드를 매번 복사·붙여넣기 한다면 유지보수가 불가능해집니다. **상속(Inheritance)**은 공통 기능을 부모 클래스(`BaseAIModel`)에 모아두고 자식 클래스가 물려받는 기술이며, **다형성(Polymorphism)**은 똑같은 `model.predict(data)` 버튼을 눌렀을 때 비전 모델은 이미지를 분류하고 LLM은 문장을 요약하도록 동작하는 마법입니다.

- **`super().__init__(...)`**: 부모 클래스의 출생 신고서를 먼저 호출해 공통 속성(이름, 버전)을 깔끔하게 세팅합니다.
- **메서드 오버라이딩(Method Overriding)**: 부모가 물려준 기본 `predict()` 메서드를 자식 클래스가 자신의 전공 분야에 맞게 재정의(덮어쓰기)합니다.

2. 실습 예제 3-1 & 3-2: 멀티모달 AI 모델 통합 추론 파이프라인

```
class BaseAIModel:
    def __init__(self, name: str, version: str):
        self.name = name
        self.version = version

    def predict(self, input_data: str) -> str:
        return f"[{self.name} v{self.version}] 기본 추론: {input_data}"

class VisionClassifier(BaseAIModel):
    def __init__(self, name: str, version: str, resolution: str):
        super().__init__(name, version)  # 부모 초기화 코드 재사용
        self.resolution = resolution

    def predict(self, input_data: str) -> str:  # 다형성 오버라이딩
        return f"[Vision {self.name} ({self.resolution})] 이미지 '{input_data}' 분석 결과 ->
'고양이 (확률 98.4%)'"

class TextSummarizerLLM(BaseAIModel):
    def __init__(self, name: str, version: str, max_tokens: int):
        super().__init__(name, version)
        self.max_tokens = max_tokens

    def predict(self, input_data: str) -> str:  # 다형성 오버라이딩
        summary = input_data[:18] + "..." if len(input_data) > 18 else input_data
        return f"[LLM {self.name} (max={self.max_tokens})] 텍스트 요약 결과 -> '{summary}'"

# 다형성의 위력: 서로 다른 모델 클래스들을 단일 for 루프로 일괄 실행!
pipeline: list[BaseAIModel] = [
    VisionClassifier("YOLO-KDT", "8.2", "640x640"),
    TextSummarizerLLM("Codex-Mini", "2.0", 512),
```

```
]
for model in pipeline:
    print(model.predict("실습 데이터 샘플 입력입니다"))
# 실행 결과:
# [Vision YOLO-KDT (640x640)] 이미지 '실습 데이터 샘플 입력입니다' 분석 결과 -> '고양이 (확률
98.4%)'
# [LLM Codex-Mini (max=512)] 텍스트 요약 결과 -> '실습 데이터 샘플 입력입니다'
```

4차시: 추상 클래스(ABC)와 데이터클래스(@dataclass) — 현대적 파이썬 OOP 끝판왕!

1. 재미있는 문법 스토리텔링: 프랜차이즈 본사 필수 규격서(ABC)와 자동 문서 생성기(@dataclass)

팀 프로젝트에서 동료 개발자가 새로운 결제 수단 클래스를 만들면서 핵심 메서드 이름을 `process_payment()` 대신 `pay_money()`라고 마음대로 지으면 전체 서버가 뻗어버립니다. 이를 막아주는 두 가지 모던 파이썬 무기를 배웁니다.

- **`ABC` & `@abstractmethod` (프랜차이즈 필수 규격서)**: "이 추상 메서드를 구현하지 않으면 아예 객체 생성조차 불허한다!"라고 강제하는 인터페이스 계약서입니다.
- **`@dataclass` (자동 문서 생성기)**: 데이터를 담는 영수증이나 DTO 클래스를 만들 때 `__init__`, `__repr__`, 비교 연산자를 자동으로 생성해 코드 길이를 절반 이하로 줄여줍니다.

2. 실습 예제 4-1: 다중 결제 게이트웨이 및 자동 정산 영수증 시스템

```
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class PaymentReceipt:
    method: str
    buyer: str
    amount: int
    fee: int
    status: str

    @property
    def net_amount(self) -> int:
        return self.amount - self.fee

class PaymentGateway(ABC):
    @abstractmethod
    def process_payment(self, buyer: str, amount: int) -> PaymentReceipt:
        """모든 결제 수단 클래스가 반드시 구현해야 하는 추상 메서드"""
        pass

class CreditCardGateway(PaymentGateway):
    def process_payment(self, buyer: str, amount: int) -> PaymentReceipt:
        fee = int(amount * 0.025)
        return PaymentReceipt("신용카드", buyer, amount, fee, "APPROVED")

class KakaoPayGateway(PaymentGateway):
```

```
    def process_payment(self, buyer: str, amount: int) -> PaymentReceipt:
        fee = int(amount * 0.01)
        return PaymentReceipt("카카오페이", buyer, amount, fee, "APPROVED")

gateways: list[PaymentGateway] = [CreditCardGateway(), KakaoPayGateway()]
for gw in gateways:
    receipt = gw.process_payment("박파이썬", 50000)
    print(f"{receipt} | 정산 금액: {receipt.net_amount:,}원")
# 실행 결과:
# PaymentReceipt(method='신용카드', buyer='박파이썬', amount=50000, fee=1250,
status='APPROVED') | 정산 금액: 48,750원
# PaymentReceipt(method='카카오페이', buyer='박파이썬', amount=50000, fee=500,
status='APPROVED') | 정산 금액: 49,500원
```

5차시: 파이썬 예외처리(try-except-else-finally) — 에러를 먹고 자라는 무적 프로그램!

1. 재미있는 문법 스토리텔링: 자율주행차의 4단계 에어백 시스템

도로 위에 작은 돌멩이(잘못된 데이터 1줄)가 있다고 해서 자율주행차가 그 자리에서 폭발(프로그램 강제 종료 `Crash`)하면 안 됩니다. 파이썬의 4단계 예외처리 구조는 돌멩이를 부드럽게 걸러내고 목적지까지 안전하게 완주하게 해줍니다.

- **`try` (위험 감지 주행 구간)**: 파일 읽기, 숫자 변환, 딕셔너리 조회 등 에러가 날 가능성이 있는 코드를 배치합니다.
- **`except` (맞춤형 에어백 전개)**: `ValueError`(타입 변환 실패), `KeyError`(필수 키 누락)처럼 에러 종류별로 정밀하게 대응합니다.
- **`else` (무사고 보너스 구간)**: `try` 블록에서 단 하나의 예외도 발생하지 않았을 때만 실행되는 안전한 후속 처리 구간입니다.
- **`finally` (블랙박스 저장 및 시동 끄기)**: 에러 발생 여부와 무관하게 **100% 무조건 실행**되어 파일 닫기나 DB 연결 해제를 보장합니다.

2. 실습 예제 5-1 & 5-2: 깨진 로그 라인을 격리하고 완주하는 무중단 AI 학습 로그 파서

```
raw_logs = [
    "epoch=1,loss=0.45,acc=88.5",
    "epoch=2,loss=invalid,acc=91.0",   # ValueError 발생 라인 ('invalid'는 float 변환 불가)
    "epoch=3,loss=0.31",               # KeyError 발생 라인 ('acc' 키 누락)
    "epoch=4,loss=0.22,acc=94.2",
]

def parse_training_logs(logs: list[str]) -> list[dict]:
    valid_records = []
    for idx, line in enumerate(logs, start=1):
        try:
            parts = dict(item.split("=") for item in line.split(","))
            record = {
                "epoch": int(parts["epoch"]),
                "loss": float(parts["loss"]),
                "acc": float(parts["acc"]),
            }
```

```
            except ValueError as e:
                print(f"[Line {idx} 변환 오류] 숫자 형식이 아닙니다: '{line}' ({e})")
            except KeyError as e:
                print(f"[Line {idx} 필드 누락] 필수 키 {e} 가 없습니다: '{line}'")
            else:
                valid_records.append(record)
                print(f"[Line {idx} 정상 수집] Epoch {record['epoch']} -> Accuracy:
{record['acc']}%")
            finally:
                pass  # 각 라인 검사 완료 후 스트림 상태 점검 위치
        print(f"[파서 종료] 총 {len(logs)}건 중 {len(valid_records)}건 정상 로드 완료.")
        return valid_records

parsed = parse_training_logs(raw_logs)
# 실행 결과:
# [Line 1 정상 수집] Epoch 1 -> Accuracy: 88.5%
# [Line 2 변환 오류] 숫자 형식이 아닙니다: 'epoch=2,loss=invalid,acc=91.0' (could not convert
string to float: 'invalid')
# [Line 3 필드 누락] 필수 키 'acc' 가 없습니다: 'epoch=3,loss=0.31'
# [Line 4 정상 수집] Epoch 4 -> Accuracy: 94.2%
# [파서 종료] 총 4건 중 2건 정상 로드 완료.
```

6차시: 사용자 정의 예외(Custom Exception)와 raise — 비즈니스 규칙을 지키는 맞춤형 경보 시스템!

1. 재미있는 문법 스토리텔링: 일반 화재경보기 vs 스마트 상황실 정밀 경보

파이썬 기본 예외(`ValueError`, `RuntimeError`)만 쓰면 "무언가 잘못됨!"이라는 사이렌만 울릴 뿐, **'일일 AI 토큰 한도가 몇 개 부족해서 실패했는지'** 즉시 파악하기 어렵습니다. `Exception` 클래스를 상속받아 우리 서비스만의 **도메인 맞춤 예외 클래스**를 만들고 `raise`로 직접 경보를 발령하면, 에러 처리 코드가 문서처럼 명확해집니다.

2. 실습 예제 6-1: Codex API 토큰 쿼터 및 프롬프트 보안 제어 시스템

```
class AIServiceError(Exception):
    """AI 서비스 최상위 기본 예외 클래스"""
    pass

class PromptEmptyError(AIServiceError):
    """프롬프트가 비어있을 때 발생하는 예외"""
    pass

class TokenQuotaExceededError(AIServiceError):
    """토큰 한도 초과 시 요청량과 잔여 한도 컨텍스트를 함께 전달하는 예외"""
    def __init__(self, requested: int, remaining: int):
        self.requested = requested
        self.remaining = remaining
        super().__init__(f"토큰 한도 초과! (요청: {requested}개, 잔여 한도: {remaining}개)")

class CodexClient:
    def __init__(self, daily_quota: int = 100):
        self.remaining_tokens = daily_quota
```

```
    def generate_code(self, prompt: str, estimated_tokens: int) -> str:
        if not prompt.strip():
            raise PromptEmptyError("프롬프트에 최소 1글자 이상의 명령어를 입력해주세요.")
        if estimated_tokens > self.remaining_tokens:
            raise TokenQuotaExceededError(estimated_tokens, self.remaining_tokens)
        self.remaining_tokens -= estimated_tokens
        return f"[생성 성공] '{prompt}' 코드 생성 완료 (남은 토큰: {self.remaining_tokens})"

client = CodexClient(daily_quota=50)
test_requests = [
    ("클래스 리팩토링 코드 작성", 30),
    ("   ", 10),
    ("대규모 프로젝트 전체 변환", 40),
]
for prompt_text, tokens in test_requests:
    try:
        result = client.generate_code(prompt_text, tokens)
        print(result)
    except PromptEmptyError as e:
        print(f"[입력 검증 실패] {e}")
    except TokenQuotaExceededError as e:
        print(f"[쿼터 경보] {e} -> 기본 요금제를 업그레이드하거나 요청 길이를 줄이세요.")
# 실행 결과:
# [생성 성공] '클래스 리팩토링 코드 작성' 코드 생성 완료 (남은 토큰: 20)
# [입력 검증 실패] 프롬프트에 최소 1글자 이상의 명령어를 입력해주세요.
# [쿼터 경보] 토큰 한도 초과! (요청: 40개, 잔여 한도: 20개) -> 기본 요금제를 업그레이드하거나 요청 길이를
줄이세요.
```

7차시: [Codex] 기반 코드 개선 & 리팩토링 실전 1 — 절차지향 스파게티를 클린 OOP로!

1. 재미있는 문법 스토리텔링: Codex에게 '시니어 아키텍트 레시피' 프롬프트 주기

Codex에게 막연히 "이 코드 좀 고쳐줘"라고 질문하면 변수 이름만 조금 바꾼 코드를 돌려줍니다. 반면 **1차시~6차시에서 배운 OOP와 예외처리 원칙을 체크리스트 형태로 프롬프트에 명시**하면, 실무 코드 리뷰를 단번에 통과하는 엔터프라이즈급 코드가 탄생합니다.

> *"프롬프트 템플릿: 아래 절차지향 함수를 리팩토링해 줘. (1) 매직 넘버(1, 2, 3)를 클래스 상수 딕셔너리로 분리할 것, (2) 주문 데이터는 @dataclass(frozen=True) 불변 객체로 캡슐화할 것, (3) 음수 가격이나 미등록 등급 입력 시 커스텀 예외(InvalidOrderError)를 raise할 것, (4) 모든 메서드에 Type Hints를 명시할 것."*

2. 실습 예제 7-1: 온라인 서점 주문/할인 계산기 Before & After 완벽 비교

[Before: 리팩토링 전 — 매직 넘버와 if-elif가 얽힌 취약한 절차지향 함수]

```
def calc_order_legacy(u_type, price, qty, coupon):
    # 문제점 1: u_type 1, 2, 3이 무슨 등급인지 작성자 외에는 알 수 없음 (매직 넘버)
    # 문제점 2: price나 qty에 음수가 들어와도 아무런 검증 없이 잘못된 금액을 계산함
    total = price * qty
    if u_type == 1:
        total = total * 0.95
```

```
    elif u_type == 2:
        total = total * 0.90
    elif u_type == 3:
        total = total * 0.80
    if coupon == "YES":
        total = total - 3000
    if total < 0:
        total = 0
    return int(total)
```

[After: Codex 리팩토링 후 — 불변 데이터클래스 + 상수 테이블 + 커스텀 예외 검증]

```
from dataclasses import dataclass

class InvalidOrderError(ValueError):
    """주문 데이터 또는 멤버십 등급이 유효하지 않을 때 발생하는 도메인 예외"""
    pass

@dataclass(frozen=True)
class OrderItem:
    name: str
    unit_price: int
    quantity: int

    def __post_init__(self):
        if self.unit_price < 0 or self.quantity <= 0:
            raise InvalidOrderError(f"유효하지 않은 가격({self.unit_price}) 또는 수량
({self.quantity})입니다.")

    @property
    def subtotal(self) -> int:
        return self.unit_price * self.quantity

class MembershipCheckout:
    DISCOUNT_RATES = {
        "BASIC": 0.05,
        "GOLD": 0.10,
        "VIP": 0.20,
    }
    COUPON_AMOUNT = 3000

    def calculate_final_price(self, item: OrderItem, tier: str, use_coupon: bool = False)
-> int:
        if tier not in self.DISCOUNT_RATES:
            raise InvalidOrderError(f"알 수 없는 멤버십 등급입니다: '{tier}'")
        discount_rate = self.DISCOUNT_RATES[tier]
        discounted = int(item.subtotal * (1.0 - discount_rate))
        if use_coupon:
            discounted -= self.COUPON_AMOUNT
        return max(0, discounted)

checkout = MembershipCheckout()
book_order = OrderItem("파이썬 클린 아키텍처", 15000, 2)
print("Before 결과:", calc_order_legacy(2, 15000, 2, "YES"))
```

```
print("After 결과:", checkout.calculate_final_price(book_order, tier="GOLD",
use_coupon=True))
# 실행 결과:
# Before 결과: 24000
# After 결과: 24000
```

8차시: [Codex] 기반 코드 개선 & 리팩토링 실전 2 — AI 데이터 전처리 파이프라인 방어적 리팩토링!

1. 실무 AI 엔지니어링 캡스톤: 더러운 데이터(Dirty Data)에서도 살아남는 전처리 아키텍처

현업 AI 프로젝트에서 수집된 원시 데이터(Raw Dataset)에는 반드시 결측치(`None`), 문자열 오타(`"N/A"`), 이상치(`120점`)가 섞여 있습니다. 마지막 8차시에서는 단 한 건의 결측치만 만나도 멈춰버리는 레거시 전처리 스크립트(Before)를 Codex와 함께 **방어적 검증기 + 예외 체이닝(`from exc`) + 전처리 통계 리포트 객체(After)**로 리팩토링하고 전후 품질 지표를 비교 분석합니다.

2. 실습 예제 8-1: AI 점수 정규화 파이프라인 Before & After 캡스톤 코드

[Before: 리팩토링 전 — 이상치·결측치 유입 시 즉시 크래시되는 취약한 함수]

```
def preprocess_legacy(data):
    res = []
    for row in data:
        # 치명적 결함: 'score' 키가 없거나 'N/A' 문자열이면 즉시 프로그램 강제 종료!
        # 치명적 결함: 100점을 초과하는 이상치(120점)도 걸러내지 못하고 1.2로 잘못 정규화함!
        score = float(row["score"])
        res.append(round(score / 100.0, 4))
    return res
```

[After: Codex 방어적 리팩토링 후 — 검증 캡슐화 + 예외 체이닝 + 자동 성공률 리포트]

```
from dataclasses import dataclass

class DataValidationError(Exception):
    """데이터 전처리 검증 실패를 나타내는 커스텀 예외"""
    pass

@dataclass
class PreprocessReport:
    normalized_scores: list[float]
    skipped_rows: list[str]

    @property
    def success_rate(self) -> float:
        total = len(self.normalized_scores) + len(self.skipped_rows)
        return round((len(self.normalized_scores) / total) * 100, 1) if total > 0 else
0.0

class SafeScorePreprocessor:
    def __init__(self, min_score: float = 0.0, max_score: float = 100.0):
        if max_score <= min_score:
```

```
            raise ValueError("max_score는 min_score보다 커야 합니다 (ZeroDivision 방지).")
        self.min_score = min_score
        self.max_score = max_score

    def _validate_and_extract(self, row: dict) -> float:
        if "score" not in row or row["score"] is None:
            raise DataValidationError("score 키가 없거나 None입니다.")
        try:
            val = float(row["score"])
        except (ValueError, TypeError) as exc:
            raise DataValidationError(f"숫자로 변환할 수 없는 값('{row['score']}')입니다.")
from exc
        if not (self.min_score <= val <= self.max_score):
            raise DataValidationError(f"허용 범위({self.min_score}~{self.max_score}) 초과:
{val}")
        return val

    def transform(self, dataset: list[dict]) -> PreprocessReport:
        normalized = []
        skipped = []
        span = self.max_score - self.min_score
        for idx, row in enumerate(dataset, start=1):
            try:
                raw_val = self._validate_and_extract(row)
            except DataValidationError as err:
                skipped.append(f"Row {idx}: {err}")
            else:
                norm_val = round((raw_val - self.min_score) / span, 4)
                normalized.append(norm_val)
        return PreprocessReport(normalized_scores=normalized, skipped_rows=skipped)

raw_dataset = [
    {"id": 1, "score": "85.0"},
    {"id": 2, "score": "N/A"},      # 변환 오류 -> 안전 격리
    {"id": 3, "score": 120},        # 범위 초과 -> 안전 격리
    {"id": 4, "score": 92.5},
    {"id": 5},                      # 키 누락 -> 안전 격리
]
processor = SafeScorePreprocessor(0.0, 100.0)
report = processor.transform(raw_dataset)
print("정규화 결과:", report.normalized_scores)
print("제외된 행 로그:", report.skipped_rows)
print(f"전처리 성공률: {report.success_rate}%")
# 실행 결과:
# 정규화 결과: [0.85, 0.925]
# 제외된 행 로그: ["Row 2: 숫자로 변환할 수 없는 값('N/A')입니다.", 'Row 3: 허용 범위(0.0~100.0) 초
과: 120.0', 'Row 5: score 키가 없거나 None입니다.']
# 전처리 성공률: 40.0%
```

3. Codex 기반 리팩토링 전후(Before vs. After) 아키텍처 품질 비교 매트릭스

평가 항목	리팩토링 전 (Before: 절차지향 레거시)	리팩토링 후 (After: Codex 기반 OOP & 방어적 설계)
가독성 및 의도 표현	매직 넘버(`1, 2, 3`)와 중첩 조건문으로 인해 비즈니스 규칙 파악이 어려움	명시적 상수 딕셔너리(`DISCOUNT_RATES`)와 `@dataclass`로 코드 자체가 문서 역할 수행
데이터 무결성 보장	음수 가격, 허용 범위 초과 점수(`120`점)가 필터링 없이 그대로 오염됨	`@property`, `__post_init__`, `_validate_and_extract`가 입구에서 오염 원천 차단
런타임 장애 복원력	단 1건의 결측치(`KeyError`)나 오타(`ValueError`)에 전체 파이프라인 즉시 중단	오류 행만 `skipped_rows`에 상세 원인과 함께 격리하고 전체 배치 작업 100% 완주
유지보수 및 확장성	신규 등급이나 결제 수단 추가 시 기존 함수를 직접 뜯어고쳐야 함 (OCP 위반)	추상 클래스(`ABC`) 상속 또는 딕셔너리 항목 1줄 추가로 안전하게 기능 확장 (OCP 준수)
디버깅 및 감사 추적	기본 스택 트레이스만 출력되어 어느 행에서 왜 실패했는지 역추적 곤란	도메인 커스텀 예외와 예외 체이닝(`from exc`), `PreprocessReport`로 즉각적인 감사 가능